SYSTEMS ARCHITECTURE & GREEN MULTI-AGENT PROTOCOLS · CANONICAL RELEASE V5.4.3

TOAI-elegant-lang: A Tri-Script Orthogonal RISC Wire Protocol, Adversarial Corpus Audit, and Empirical Comprehension Harness for Multi-Agent Communication

Koji Toriyama (PhenoX) · Principal Systems Architect, PhenoX AI Alliance, Kawasaki, Japan

Repository & Reproducibility Artifact: https://github.com/PhenoX-AI-Alliance/TOAI-elegant-lang | **PyPI:** `pip install toai-elegant-lang==5.4.3`

ABSTRACT

Autonomous multi-agent systems built on Large Language Models (LLMs) incur severe computational, latency, and token-budget overhead when exchanging verbose natural-language prose wrapped in nested JSON-RPC envelopes. Compressing agent-to-agent (A2A) messages exposes a fundamental systems trilemma among *wire token efficiency*, *syntactic and numeric type fidelity*, and *human auditability*. This paper presents **TOAI-elegant-lang (v5.4.3)**, a zero-dependency, deterministic compiler and wire protocol inspired by the Rosetta Stone that aligns multi-agent state transitions across three synchronized representations: (1) **Demotic Wire Script**, a compact causal wire syntax (`@CTX:... :: §S01 ... §X64`); (2) **Sacred Expanded Script**, a deterministic macro- and primitive-expanded canonical form (`§GAIA_WARMTH_HARMONY`); and (3) the **Rosetta Audit Layer**, which attaches natural-language definitions and domain invariants for human inspection. Under `o200k_base` and `cl100k_base` Byte-Pair Encoding (BPE) tokenizers, each compact §-prefixed identifier occupies 2–3 BPE tokens (`'§'` + `Prefix` + `Digits`, or 2 tokens with ASCII prefixes), while the 256-primitive + 256-macro codebook fits inside a ∼1,000-token system prompt header. Evaluated on the reproducible multi-format benchmark (`benchmarks/fair_bench.py`), Demotic Wire averages **73.2 BPE tokens/message**, achieving a **79.1% token reduction vs. Verbose A2A JSON** (349.8 tokens), **55.7% vs. Structured Tool JSON** (165.2 tokens), and **41.0% vs. Minified Key JSON** (124.2 tokens) while preserving 100.0% lossless syntactic round-trip reconstruction and amortizing its 1,000-token dictionary header within **4–20 messages uncached** or **1–2 messages under 90% prompt caching**. Following multi-AI adversarial auditing, we introduce a non-circular **Independent Corpus Alignment Auditor** (`src/audit_corpus_alignment.py`) validated against a stratified human gold-standard sample ($n = 60$, raw agreement $p_o = 93.33\%$, **Cohen's** $\kappa = 0.8611$ [95% CI: 0.7296–0.9926]), raising corpus scenario-to-wire alignment from 22.67% (364/1,606) to 100.0% (1,606/1,606). Finally, we separate syntactic round-trip fidelity from receiver semantic comprehension via a 5-condition statistical harness (`benchmarks/eval_comprehension.py` with Wilson 95% CIs, exact McNemar tests, and 4-way option rotation), reporting provisional blind comprehension results ($n = 23$, 4-choice chance 25%: Condition A Wire-only 12/23 = 52% [95% CI: 33–71%]; Condition B Sacred names 16/23 = 70% [95% CI: 49–84%]) alongside an unobservable internal-counter EUF-CMA cryptographic attestation proof.

1. Introduction

As Large Language Model (LLM) applications evolve from isolated single-turn assistants into cooperative multi-agent workflows—such as Anthropic's Model Context Protocol (MCP) [1], Google's Agent-to-Agent (A2A) JSON-RPC 2.0 orchestrators [2], and multi-agent conversation frameworks such as AutoGen [5]—inter-agent communication overhead has emerged as a primary bottleneck in inference cost, context-window saturation, and key-value (KV) cache memory footprint. In conventional agent topologies, autonomous nodes coordinate state transitions by generating explanatory natural-language prose embedded inside deeply nested JSON schemas. While human-readable, repeated schema keys, conversational filler, and JSON punctuation delimiters consume a disproportionate share of the token budget and scale prefill attention FLOPs quadratically with sequence length.

Attempting to compress inter-agent messages exposes a strict **Multi-Agent Communication Trilemma**:

1. **Wire Token Efficiency:** Minimizing BPE subword tokens per state transition and amortizing any shared protocol specification across multi-turn sessions.
2. **Syntactic, Numeric, and Causal Fidelity:** Guaranteeing 100% deterministic preservation of structured key-value parameters (e.g., leading-zero string identifiers `"007"`, embedded delimiters `"a, b"`, floating-point thresholds `150.50`), causal transition probabilities $p \in [0, 1]$, transition costs $c \in [0, 0.9999]$, and fault-fallback edges (`!->`).
3. **Human Auditability and Cross-Model Portability:** Ensuring that human operators and heterogeneous LLM families (GPT, Claude, Gemini, Qwen, Llama, Gemma) can inspect, verify, and exchange packets over standard text APIs without opaque binary blobs or model-specific hidden-state adapters.

Existing approaches sacrifice at least one vertex of this trilemma. Heuristic perplexity-based prompt pruning such as LLMLingua [4] drops tokens based on language-model perplexity, which risks corrupting JSON syntax delimiters or numeric literals in structured payloads. Conversely, reinforcement-learned (RL) discrete emergent protocols [6, 7] or continuous latent-vector handoffs sacrifice human auditability and cannot traverse commercial text-only REST APIs across heterogeneous model families. Flat key-value templates (`KEY=VAL`) reduce token count but discard the causal execution graph and fault-recovery semantics.

To address this trilemma with engineering rigor and empirical transparency, we present **TOAI-elegant-lang (v5.4.3)**, a zero-dependency Python compiler and wire protocol built on an orthogonal Reduced Instruction Set Computer (RISC) instruction architecture and a Tri-Script Rosetta design. Crucially, release `v5.4.3` incorporates lessons from an intensive multi-AI adversarial audit (conducted across independent ChatGPT and Claude execution sandboxes), replacing idealized assumptions with strictly reproducible, self-contained evaluation tools. Our contributions are fourfold:

1. **8-Bit Orthogonal RISC & Unified Tri-Script Architecture:** We formalize a three-layer representation aligning *Demotic Wire Script* (compact opcodes `§S01..§X256`), *Sacred Expanded Script* (canonical uppercase identifiers `§[A-Z][A-Z0-9_]*`), and the *Rosetta Audit Layer* (human-readable definitions). We provide an exact BPE subword accounting showing that §-prefixed opcodes occupy 2–3 BPE tokens (`'§'` + `Prefix` + `Digits`) under `o200k_base` and `cl100k_base`.
2. **Quote-Aware Stateful LL(1) Parser & Transparent OOV Auditing:** We specify a conflict-free LL(1) Extended Backus-Naur Form (EBNF) grammar paired with a character-by-character quote-aware payload scanner in `src/codec.py` that losslessly preserves quoted strings (`"007"`, `"a, b"`, `"x]y"`, escaped quotes) and enforces explicit out-of-vocabulary policies (`on_oov="error" | "keep" | "nearest"`) with a complete `oov_substitutions` audit log.
3. **Non-Circular Corpus Alignment Auditor & Fair Break-Even Benchmark:** To eliminate circular validation between corpus generation and evaluation, we build an independent deterministic auditor (`src/audit_corpus_alignment.py`) governed

by domain-specific semantic whitelists and cross-domain bleed blacklists. Validated against a stratified 60-item human audit (**Cohen's** $\kappa = 0.8611$), the auditor detects and repairs historical scenario-to-wire mismatches (raising alignment from 22.67% to 100.0%). We pair this with `benchmarks/fair_bench.py`, quantifying token savings and header break-even points across five representations.

4. **Statistical Receiver Comprehension Harness & EUF-CMA Attestation:** We strictly distinguish *syntactic round-trip losslessness* (100.0%) from *blind receiver semantic comprehension*. We introduce `benchmarks/eval_comprehension.py` (implementing Conditions A–E, Shuffle Control, Wilson 95% CIs, exact McNemar tests, and 4-way option rotation) and report provisional blind evaluation results alongside a formal EUF-CMA security proof for internal-counter HMAC-SHA256 origin attestation.

2. Related Work and Architectural Positioning

We position TOAI-elegant-lang relative to four established paradigms in agent communication and prompt compression, restricting all comparisons to verifiable specifications, peer-reviewed literature, and deterministic baselines implemented directly in our repository (Table 1):

- **Classical Agent Communication Languages (FIPA-ACL [3]) and Modern JSON-RPC Orchestrators (MCP [1], A2A [2], AutoGen [5]):** FIPA-ACL standardized performative-based agent messaging (`inform`, `request`, `propose`) using S-expressions or XML. Modern LLM ecosystems such as Anthropic MCP [1] and Google A2A [2] adopt JSON-RPC 2.0 envelopes, while multi-agent frameworks such as AutoGen [5] exchange natural-language turns wrapped in role-tagged JSON dictionaries. While universally portable across text APIs, full JSON-RPC and prose explanations incur high token overhead (349.8 tokens/message in our benchmark), and even stripped-down *Structured Tool JSON* (165.2 tokens) or *Minified Key JSON* (124.2 tokens) repeatedly pay BPE tokenization costs for structural braces, quotes, colons, and field names on every turn.
- **Binary Serialization Formats (Protocol Buffers [8]):** Google's Protocol Buffers and MessagePack achieve compact byte-level encoding over raw TCP sockets. However, LLM text completion APIs cannot ingest raw arbitrary binary octets (`0x00--0xFF`) directly inside prompt context windows without Base64 ASCII armoring or external tool-call indirection. Base64 encoding inflates byte length by +33.3% and produces high-entropy alphanumeric strings that fragment into 1–2 characters per BPE subword while stripping human readability on the wire.
- **Perplexity-Based Prompt Compression (LLMLingua [4]):** Jiang et al. [4] introduced coarse-to-fine prompt compression using a small causal language model to prune low-perplexity tokens before inference. While highly effective for summarizing long natural-language documents or RAG contexts, applying stochastic or perplexity-driven token dropping to structured inter-agent packets risks deleting syntactic delimiters (brackets, quotes, colons) or altering numerical parameters, and requires an auxiliary neural forward pass.
- **Emergent Discrete and Continuous Multi-Agent Communication (Foerster et al. [6]; Lazaridou et al. [7]):** Deep multi-agent reinforcement learning (MARL) systems learn emergent discrete symbol policies (DIAL/RIAL [6]) or continuous hidden-state projections [7] to coordinate tasks. However, emergent codebooks are tied to the training environment's latent space, lack deterministic human-auditable semantics, and suffer from semantic drift or vocabulary mismatch when transferred across independently trained commercial LLM families.

Table 1: Architectural Comparison of Multi-Agent Communication Paradigms (Verified Standards & Repository Baselines)

Communication Paradigm	Wire Encoding	Causal AST & Guards	Deterministic Round-Trip	Cross-API Portability	Human Auditability & Operational Characteristics
Verbose A2A / MCP JSON-RPC [1, 2, 5]	Pretty JSON + NL Prose	Yes (Verbose)	100.0%	100% (Text API)	High readability; highest token cost (349.8 tok/msg; baseline).
Structured Tool JSON (`fair_bench.py`)	Compact JSON Schema	Yes (JSON Objects)	100.0%	100% (Text API)	Lossless structure; repeated JSON keys/syntax (165.2 tok/msg).
Minified Key JSON (`fair_bench.py`)	Short-Key JSON (`"c","s"`)	Yes (Abbrev. JSON)	100.0%	100% (Text API)	Abbreviated keys require schema lookup (124.2 tok/msg).
Simplified Flat Template (`KEY=VAL`)	Delimited Flat String	**No (Lossy)**	Lossy (No AST)	100% (Text API)	Lowest raw tokens (34.5 tok/msg), but loses causal edges & fallbacks.
Binary Protobuf [8] (Base64 Text Wire)	Base64-Armored Octets	Schema-Defined	100.0% (External)	Requires Tool/Decode	Opaque Base64 string in prompt context; BPE subword fragmentation.
Perplexity Token Pruning (LLMLingua [4])	Pruned Natural Text	Unstructured	Lossy / Approximate	100% (Text API)	Designed for prose/RAG; requires scorer model; can drop JSON syntax.
Emergent MARL Codes [6, 7]	Learned IDs / Vectors	Implicit in Policy	Model-Dependent	Homogeneous Only	Opaque latent symbols; not directly interpretable by humans.
TOAI-elegant-lang (v5.4.3 Ours)	**Demotic RISC Wire (`§ID`)**	**Yes (Native LL(1) AST)**	**100.0% Lossless**	**100% (Text API)**	**Tri-Script (Demotic / Sacred / Rosetta); 73.2 tok/msg; $\mathcal{O}(L)$ CPU.**

3. System Architecture, LL(1) Grammar, and Quote-Aware Stateful Codec

3.1 Tri-Script Architecture and 8-Bit Orthogonal RISC Lexicon Hierarchy

TOAI-elegant-lang resolves the tension between wire compactness and human auditability by decoupling the machine-transmitted representation from the human-inspected representation via three deterministic layers (Table 2):

1. **Demotic Wire Script (Compact Machine Layer):** The compact wire format transmitted between agents. It combines a context header (`@CTX:DOMAIN`), 1-byte logical core opcodes (`§S01`–`§H32` across 8 semantic categories: `S` System, `D` Data, `G` Governance/Protocol, `F` Fault/Finance, `E` Execution, `A` Audit, `C` Communication/Gaia, `L` Logic, plus `H` Harmony aliases totaling exactly 256 core primitives in `core_lexicon.json`), 256 active higher-order macro opcodes (`§X01`–`§X969` managed

via a Least Recently Used [LRU] window in `macro_lexicon.json`), bracketed key-value payloads (`[k:v]`), and causal/fault operators (`-[p:..., c:...]->`, `=>`, `!->`).

2. **Sacred Expanded Script (Canonical Symbolic Layer):** A deterministic expansion generated by `codec.decode(wire)` in which every compact opcode and macro is expanded into its canonical uppercase snake-case identifier matching the strict regular expression `^§[A-Z][A-Z0-9_]*$` (e.g., expanding `§X64` into `(§GAIA_WARMTH_EXCHANGE > { §SYS_RESOURCE_ARBITRAGE_THROTTLE } => §SYS_LOAD_FALLBACK)` and `§G48` into `§PROTO_HEALTH_ATTEST`).
3. **Rosetta Audit Layer (Human Governance Layer):** Stored in `rosetta_layer.json` and `core_lexicon.json["definitions"]`, this layer maps every primitive and macro to a concise English and Japanese specification for compliance auditing and Condition C glossary injection.

Honest BPE Subword Tokenization Analysis: Unlike raw bitstreams, LLM APIs bill and allocate KV-cache memory in discrete BPE subword tokens. Under OpenAI's `o200k_base` (GPT-4o) and `cl100k_base` (GPT-4 / Claude-compatible) tokenizers, as well as our deterministic fallback estimator `count_bpe_tokens()` in `src/codec.py`, the non-ASCII section sign `'§'` (`U+00A7`) is tokenized separately from the following ASCII domain letter and two-digit number. Consequently, every canonical §-prefixed identifier (§S01, §G48, §C20, §X64, §X354) consumes **3 BPE tokens** (2–3 **tokens: `'§'` + `Prefix` + `Digits`)**, whereas substituting an ASCII prefix (`@S01` or `#S01`) merges the prefix and letter to resolve in 2 BPE tokens. Even at 3 BPE tokens per §-opcode, collapsing multi-primitive subgraphs into single §X macros and replacing verbose JSON keys with LL(1) causal arrows reduces whole-packet token count by 41.0%–79.1% against lossless JSON baselines (Section 5).

Table 2: Tri-Script Representation and Lexicon Hierarchy in TOAI-elegant-lang v5.4.3

Architectural Layer	Source / Capacity	Canonical Pattern / Example	Validation & Runtime Role in v5.4.3
1. Demotic Wire Script (Core Primitives)	`core_lexicon.json` (256 Primitives)	§S01..§L32, §G48, §C20 (3 BPE tok/symbol)	Immutable 256-primitive base vocabulary; validated at startup and by `test_17` for bijective mapping.
1b. Demotic Wire Script (Active LRU Macros)	`macro_lexicon.json` (256 Active + Archive)	§X01..§X969 (e.g., §X64, §X354; 3 BPE tok/symbol)	Depth-1 macro expansion into core primitives; pinned core macros prevent LRU thrashing.
2. Sacred Expanded Script (Canonical AST)	`codec.decode()` Output	`^§[A-Z][A-Z0-9_]*$` (e.g., §GAIA_WARMTH_HARMONY)	Zero lowercase or CJK corruption permitted; strictly enforced by unit test `test_17` across all files.
3. Rosetta Audit Layer (Human Definitions)	`rosetta_layer.json` & `definitions`	Natural-language semantic specification per symbol	Used for human governance inspection, `audit_corpus_alignment.py`, and `eval_comprehension.py --cond C`.
Inline Structured Payload (`[k:v]`)	Quote-Aware Scanner (`_parse_payload_block`)	`[pair:"USD/JPY", stop:150.5, code:"007", note:"a, b"]`	Stateful character scanner preserving quoted strings, leading zeros, commas, brackets, and escapes.

3.2 Conflict-Free LL(1) EBNF Grammar and Quote-Aware Stateful Payload Parser

To guarantee deterministic $\mathcal{O}(L)$ parsing without regular-expression backtracking vulnerabilities or silent data corruption, `toai-elegant-lang v5.4.3` defines a conflict-free LL(1) grammar paired with a stateful character-level payload parser in `src/codec.py`:

```
(* Canonical LL(1) EBNF Grammar & Lexical Terminals for TOAI-elegant-lang v5.4.3 *)
packet          = [ context_header, "::" ], statement_chain ;
context_header  = "@CTX:", identifier ;
statement_chain = expr_node, { operator, expr_node } ;
expr_node       = ( "(", statement_chain, ")" | "{", statement_chain, "}" | action_node ), [ payload_block ] ;
action_node     = token_id, [ payload_block ], [ guard_block ] ;
token_id        = "§", ( risc_id | macro_id | sacred_name ) ;
payload_block   = "[", kv_pair, { ",", kv_pair }, "]" ;
kv_pair         = identifier, ":", ( quoted_string | float_lit | int_lit | bool_null_id ) ;
guard_block     = "?>", "{", condition_expr, "}" ;
operator        = causal_arrow | seq_arrow | fault_arrow | ">" | "?" ;
causal_arrow    = "-[", [ "p:", float_val, "," ], ( "c:" | "cost:" ), cost_mantissa, "]->" ;
seq_arrow       = "=>" ;   fault_arrow = "!->" ;
sacred_name     = upper_alpha, { upper_alpha | digit | "_" } ;  (* Strict ^§[A-Z][A-Z0-9_]*$ *)
quoted_string   = '"', { ( utf8_char - ( '"' | '\' ) ) | '\"' | '\\' | '\n' | '\t' }, '"' ;
```

Eliminating Naive Split Bugs via Stateful Quote-Aware Scanning: During our external adversarial code audit, sandbox testing revealed that naive payload parsers using `str.split(",")` or unanchored regexes `r"\[(.*?)\]"` corrupt structured payloads whenever string values contain commas (`note:"a, b"`), closing brackets (`expr:"x]y"`), escaped quotes (`msg:"say \"hi\""`), or leading-zero identifiers (`code:"007"` coerced into integer 7). In `v5.4.2+` and `v5.4.3`, `ElegantCodec._extract_outer_blocks()` and `ElegantCodec._parse_payload_block()` implement a deterministic finite-state character scanner tracking `in_quotes` and `escape` flags:

- **Type-Exact Scalar Distinction:** Any value enclosed in double quotes (`"007"`) is flagged with `__quoted__ = True` and preserved strictly as a Python `str` (`"007"`), whereas unquoted numeric tokens (`007`, `42`, `150.5`) are parsed as `int` or `float`.
- **Delimiter Immunity Inside Quotes:** Commas (`,`), colons (`:`), brackets (`[`, `]`), and §-prefixed substrings inside quoted string literals are never split or mutated during `encode()` or `decode()` (verified by `test_07` through `test_10` and `test_15` in `tests/test_codec.py`).
- **Explicit Out-Of-Vocabulary (`on_oov`) Policy & Audit Trail:** Rather than silently mapping unknown symbols to a fallback code such as §G01 or §OMEGA (which masks failures and inflates apparent compression), `codec.encode(text, on_oov=...)` enforces three explicit modes: (i) `on_oov="error"` (default), which immediately raises `UnknownSymbolError`; (ii) `on_oov="keep"`, which leaves unregistered symbols untouched on the wire; and (iii) `on_oov="nearest"`, which resolves to the closest domain primitive via token-overlap scoring while recording every replacement in `EncodedWireResult.substitutions` and `benchmark_compression()["oov_substitutions"]` (verified by `test_11`–`test_14`).

4. Multi-AI Adversarial Audit & Independent Corpus Alignment ($\kappa = 0.8611$)

4.1 Full Disclosure of Multi-AI Adversarial Audit Findings (v5.4.1 → v5.4.3)

To ensure scientific integrity, `toai-elegant-lang` was subjected to blind black-box and white-box auditing by independent AI systems (OpenAI ChatGPT code-execution sandbox and Anthropic Claude comprehension & statistical review) prior to release `v5.4.3`. Table 3 discloses every vulnerability identified during these audits and its permanent engineering resolution verified by our 20-test automated suite (`tests/test_codec.py` and `src/test_telepath_integrity.py`).

Table 3: Summary of Multi-AI Adversarial Audit Findings and Engineering Resolutions in Canonical Release v5.4.3

Audit Defect ID	Identified Vulnerability / Discrepancy	Deterministic Engineering Resolution in v5.4.3 & Unit Test
Defect 1: Wheel Packaging & Silent Dict Fallback	PyPI wheel omitted `lexicon/*.json`, and `ElegantCodec` silently initialized an empty dict `{}` when files were missing.	Packaged `src/lexicon/*.json` inside `pyproject.toml` wheel data; `ElegantCodec` now raises `FileNotFoundError` / `RuntimeError` on missing or empty lexicons (`test_01`–`test_04`).
Defect 2: Payload Delimiter & `"007"` Coercion	Naive regex/comma splitting broke on `note:"a, b"`, `expr:"x]y"`, and coerced quoted `code:"007"` to integer `7`.	Implemented stateful character-by-character quote/escape scanner in `_parse_payload_block()` and `_protect_quoted_spans()` (`test_07`–`test_10`, `test_15`).
Defect 3: Silent OOV Fallback to `§G01/§OMEGA`	Unregistered symbols were silently rewritten to `§G01` or `§OMEGA`, masking OOV failures in benchmarks.	Default `on_oov="error"` raises `UnknownSymbolError`; `"keep"` preserves symbol; `"nearest"` logs every fallback in `oov_substitutions` (`test_11`–`test_14`).
Defect 4: Naive Whitespace Token Estimator	Fallback token counter `r"\w+|\S"` undercounted BPE subwords when `tiktoken` was offline and hid estimation status.	Implemented CamelCase/snake_case/digit-chunk BPE subword estimator in `count_bpe_tokens()` returning explicit `is_estimated: bool` (`test_16`).
Defect 5: Lexicon & Corpus Symbol Typos (`GAiar`)	Mixed-case/corrupted identifiers (`§GAiar_WARMTH_HARMONY`, `§HARMONIC_ERECVERY_NEXUS`, CJK `U+822C/U+7684`, `_v2`) leaked into Q11/Q16 prompts.	Normalized all 256 primitives, 256 macros, Rosetta mappings, and 1,606 corpus records to strict `^§[A-Z][A-Z0-9_]*$` with zero undefined symbols (`test_17`).
Defect 6: Corpus Scenario-to-Wire Misalignment	77.33% of historical records in `corpus/dialogues.jsonl` had scenario labels (e.g., FX Hedge Q3/Q9, Parser Bugfix Q18) mismatched with wire symbols.	Built non-circular auditor `src/audit_corpus_alignment.py` ($\kappa = 0.8611$ vs. human gold standard) and repaired all 1,606 records to 100.0% alignment (`test_18`).

4.2 Non-Circular Independent Corpus Alignment Audit and Human Agreement ($\kappa = 0.8611$)

During the blind comprehension audit (Section 6), inspecting individual items revealed a critical data quality issue in the historical evolution log (`corpus/dialogues.jsonl`, $N = 1{,}606$ records):

- **Recovery Pattern Leak (e.g., Q3, Q9):** Records labeled with scenario *"Emergency FX hedging under rapid exchange-rate shift"* contained compact wires composed exclusively of fault-recovery primitives (`§PROTO_REPLAY_GUARD`, `§PROTO_HEALTH_ATTEST`, `§PROTO_GRACEFUL_DRAIN`, `§SAFE_FALLBACK_*`) with zero financial, exchange, or risk primitives.
- **Cross-Domain Bleed (e.g., Q18):** Records labeled *"Syntax parser bugfix task"* contained macro `§X64` (`GAIA_WARMTH_ARBITRAGE_FALLBACK`, expanding to `§SYS_RESOURCE_ARBITRAGE_THROTTLE`), causing receiver LLMs to rationally classify the wire as financial arbitrage rather than code debugging.

Preventing Circular Validation and Data Leakage: Two methodological pitfalls must be avoided when auditing and repairing a benchmark corpus: (1) relabeling scenarios based on the evaluated LLM's answers constitutes *test-set leakage* and artificially inflates comprehension accuracy; and (2) auditing symbol-to-scenario alignment using the same dictionary `category` field or keyword lookup table used by the corpus generator creates a *circular reference* that misses the very mismatches produced by the generator.

To guarantee independence, `src/audit_corpus_alignment.py` defines a standalone **Independent Audit Ontology** (`INDEPENDENT_AUDIT_ONTOLOGY`) that is completely decoupled from `core_lexicon.json["category"]` and from all generation specifications. For each of the 7 operational scenario domains (`FX`, `CODE_PARSER`, `LATENT_SYNC`, `TASK_BUDGET`, `HYPOTHESIS`, `GAIA_HARMONY`, `RECOVERY`), the auditor expands every macro to its Sacred primitive identifiers and evaluates four deterministic rules:

1. **Plaintext Contamination Check:** Rejects any wire containing raw Japanese characters or natural-language prose outside `[k:v]` literals as `MISMATCH (Plaintext Leak)`.
2. **Cross-Domain Bleed Blacklist:** Rejects any record whose expanded Sacred symbols match a domain-forbidden token pattern (e.g., `ARBITRAGE`, `FX_`, or `PORTFOLIO` inside `CODE_PARSER`, `LATENT_SYNC`, or `HYPOTHESIS`) as `MISMATCH (Domain Bleed)`.
3. **Recovery Pattern Leak Check:** For any non-`RECOVERY` scenario, if $> 50\%$ of expanded primitives match pure cluster-recovery signatures (`REPLAY_GUARD`, `GRACEFUL_DRAIN`, `PARTIAL_COMMIT`, `CIRCUIT_BREAKER`, `SAFE_FALLBACK`) and < 2 domain-specific anchor primitives are present, the record is flagged as `MISMATCH (Recovery Pattern Leak)`.
4. **Required Domain Anchor Whitelist:** Requires at least `min_domain_hits` semantic anchor tokens specific to the scenario domain, otherwise flagging `MISMATCH (Missing Domain Primitives)`.

Human Gold-Standard Validation ($n = 60$) and Cohen's κ: To validate the reliability of the mechanical auditor itself prior to corpus repair, we extracted a stratified sample of $n = 60$ pre-fix corpus records across all 7 domains (`python src/audit_corpus_alignment.py --export-sample 60 --out corpus/human_audit_sample.csv`) and compared the mechanical verdict (`MATCH` vs. `MISMATCH`) against independent human inspection (`--score-sample corpus/human_audit_sample.csv`). The resulting 2×2 confusion matrix recorded **22 True Matches**, **34 True Mismatches**, **2 False Positives**, and **2 False Negatives**, yielding a raw observed agreement of $p_o = 56/60 = \mathbf{93.33}\%$ over a chance agreement baseline of $p_e = 52.00\%$, and a **Cohen's Kappa of** $\kappa = 0.8611$ (SE $= 0.0671$, 95% CI: $[0.7296, 0.9926]$, "almost perfect agreement"). Table 4 reports the pre-fix and post-fix alignment rates across all 1,606 records in `corpus/dialogues.jsonl`.

Table 4: Independent Corpus Alignment Audit Across All $N = 1{,}606$ Records in `corpus/dialogues.jsonl` (`src/audit_corpus_alignment.py`)

Scenario Domain Category	Records N	Pre-Fix Aligned	Post-Fix Aligned	Primary Pre-Fix Failure Mode Identified by Independent Ontology
1. `CODE_PARSER` (Syntax/AST Bugfix)	234	4 (1.71%)	**234 (100.0%)**	Domain Bleed (`§X64 ARBITRAGE` macro) & Recovery Pattern Leak
2. `FX` (Currency Hedge & Risk)	245	5 (2.04%)	**245 (100.0%)**	Recovery Pattern Leak (`REPLAY_GUARD` / `GRACEFUL_DRAIN` only)
3. `GAIA_HARMONY` (Earth/Co-Prosperity)	228	98 (42.98%)	**228 (100.0%)**	Recovery Pattern Leak & Plaintext Leak in legacy evolution records
4. `HYPOTHESIS` (Parallel Path Verify)	192	0 (0.00%)	**192 (100.0%)**	Missing Deductive/Bayesian Anchors & `§X64` Arbitrage Bleed
5. `LATENT_SYNC` (Tensor Map Sync)	241	29 (12.03%)	**241 (100.0%)**	Missing Tensor/Vector/Manifold Anchors & Recovery Pattern Leak
6. `RECOVERY` (Fault & Circuit Drain)	245	128 (52.24%)	**245 (100.0%)**	Domain Bleed (`ARBITRAGE` macro used inside pure cluster drain)
7. `TASK_BUDGET` (Token Quota Alloc.)	221	18 (8.14%)	**221 (100.0%)**	Recovery Pattern Leak (missing resource/quota/budget primitives)
Total / Corpus Aggregate	**1,606**	**364 (22.67%)***	**1,606 (100.0%)**	**Human vs. Machine Audit** ($n = 60$): $p_o = 93.33\%$, **Cohen's** $\kappa = 0.8611$

*Note: 82 legacy records with uncategorized free-form scenario labels aligned prior to domain normalization ($282 + 82 = 364/1{,}606 = 22.67\%$ total pre-fix alignment); post-fix corpus spans 42 distinct scenario variants across the 7 canonical domains with 1,589 unique usable wires.

5. Empirical Evaluation I: Fair Multi-Format BPE Benchmark & Header Amortization

To avoid inflated compression claims that compare compact wire codes exclusively against verbose natural-language essays, `benchmarks/fair_bench.py` evaluates **TOAI Demotic Wire** against four realistic alternative representations across four representative multi-agent coordination scenarios (`S1_FX_HEDGE`, `S2_DIST_RECOVERY`, `S3_TENSOR_ARBITRAGE`, and `S4_PUB_SEAL_AUDIT`):

1. **Verbose A2A (Baseline):** Standard natural-language intent prose + semantic specifications + full JSON causal graph.
2. **Structured Tool JSON:** Lossless minified JSON (`separators=(",",":")`) omitting all prose explanations and retaining only structured pipeline steps, parameters, causal transitions `{"p":0.99,"c":0.01}`, and fallback branches.
3. **Minified Key JSON:** Aggressive short-key JSON (`{"c":"FX_HEDGE","s":[{"a":"X64","p":{...},"t":[0.99,0.01]}]}`) using single-character keys and compact opcode identifiers inside JSON syntax.
4. **Simplified Lossy Template:** Flat `DOMAIN|k1=v1,k2=v2` string template. While requiring only 34.5 tokens, it is *structurally lossy*: it drops intermediate pipeline opcodes, causal transition weights (p, c), and conditional fault-recovery branches (`!->`, `?`).
5. **TOAI Demotic Wire:** Canonical wire output of `ElegantCodec().encode(sacred_verbose, on_oov="error")`, preserving the full causal AST, all typed parameters, and fault-recovery branches in **73.2 BPE tokens**.

Session Header Amortization (Break-Even Analysis): In zero-shot cross-model communication where the receiver does not have the codebook pre-loaded, transmitting the active dictionary header costs $H \approx 1{,}000$ BPE tokens at session start. Given per-message token savings $\Delta_{\text{tok}} = T_{\text{baseline}} - T_{\text{wire}}$, the session break-even message count is:

$$N^{\text{std}}_{\text{break-even}} = \left\lceil \frac{H}{T_{\text{baseline}} - T_{\text{wire}}} \right\rceil, \qquad N^{\text{cache (90\%)}}_{\text{break-even}} = \left\lceil \frac{0.10 \cdot H}{T_{\text{baseline}} - T_{\text{wire}}} \right\rceil \tag{1}$$

As shown in Table 5 and Table 6, TOAI Demotic Wire achieves net token savings after **4 messages** against Verbose A2A, **11 messages** against Structured Tool JSON, and **20 messages** against Minified Key JSON in uncached sessions—and breaks even within **1 to 2 messages** when provider prompt caching (90% discount on cached system prefixes) is enabled.

Table 5: Reproducible Multi-Format Tokenization Benchmark (`python benchmarks/fair_bench.py`, $N = 4$ Multi-Agent Scenarios)

Format Representation	Avg Chars	o200k Tok	cl100k Tok	vs. Wire Diff	Causal Graph	Per-Scenario Token Breakdown (S1 / S2 / S3 / S4)
1. Verbose A2A (Baseline)	870.8	349.8	349.8	**−79.1% saved**	Yes (Prose+JSON)	346 / 391 / 348 / 314 tok
2. Structured Tool JSON	288.2	165.2	165.2	**−55.7% saved**	Yes (Full JSON)	145 / 225 / 159 / 132 tok
3. Minified Key JSON	170.2	124.2	124.2	**−41.0% saved**	Yes (Short Keys)	105 / 172 / 118 / 102 tok
4. Simplified Template	75.2	34.5	34.5	+112.3% more	**No (Lossy Flat)**	24 / 49 / 39 / 26 tok (Drops AST)
5. TOAI Demotic Wire (v5.4.3)	**132.8**	**73.2**	**73.2**	**BASELINE**	**Yes (Causal AST)**	**59 / 100 / 71 / 63 tok (Lossless)**

Table 6: Session Dictionary Header Amortization / Break-Even Analysis (System Dictionary Header $H = 1{,}000$ BPE Tokens, Eq. 1)

Comparison Target Baseline	Target Tok/Msg	Wire Tok/Msg	Saved / Msg	Break-Even (Std)	Break-Even (90% Prompt Cache)
Verbose A2A (Baseline)	349.8 tok	73.2 tok	**276.5 tok**	**4 msgs** ($\geq 1{,}106$ tok saved)	**1 msg** (Effective header = 100 tok)
Structured Tool JSON (Lossless)	165.2 tok	73.2 tok	**92.0 tok**	**11 msgs** ($\geq 1{,}012$ tok saved)	**2 msgs** (Effective header = 100 tok)
Minified Key JSON (Short Keys)	124.2 tok	73.2 tok	**51.0 tok**	**20 msgs** ($\geq 1{,}020$ tok saved)	**2 msgs** (Effective header = 100 tok)

6. Empirical Evaluation II: Receiver Blind Comprehension Harness (`eval_comprehension.py`)

6.1 Experimental Hygiene, Five Blind Conditions (A–E), and Statistical Methodology

A central methodological contribution of `v5.4.3` is separating **deterministic codec round-trip accuracy** (`codec.decode(codec.encode(x)) == x`, which is 100.0% by algebraic construction) from **zero-shot receiver LLM semantic comprehension** (whether an LLM receiving a wire packet with `@CTX` and `[k:v]` stripped can identify the operational scenario among 4 choices at `temperature` = 0). Building upon the initial evaluation design by Anthropic Claude, `benchmarks/eval_comprehension.py` enforces strict experimental hygiene across five conditions:

- **Condition A (Compact Wire Skeleton Only):** Both `@CTX:DOMAIN` and all `[k:v]` payload blocks are stripped, leaving only compact opcodes and causal operators (e.g., `§X64 -[p:0.99, c:0.01]-> §G48 => §C20`). Tests whether an LLM can infer intent purely from topology and opaque §-codes (21.5 BPE tokens/item).
- **Condition B (Sacred Expanded Names Only):** All macros and primitives are deterministically expanded to their uppercase Sacred identifiers (e.g., `(§GAIA_WARMTH_EXCHANGE > { §SYS_RESOURCE_ARBITRAGE_THROTTLE } => §SYS_LOAD_FALLBACK) -[p:0.99, c:0.01]-> §PROTO_HEALTH_ATTEST => §GENTLE_RESOURCE_SHARING`), with `@CTX` and `[k:v]` stripped (48.0 BPE tokens/item). Directly tests the semantic transparency of Sacred naming.
- **Condition C (Sacred Expanded + Rosetta Glossary):** Condition B augmented with a per-item `GLOSSARY` block listing the natural-language definitions of all primitives and macros present in the message (152.0 BPE tokens/item).
- **Condition D (Shuffle Control – Negative Control):** Preserves the exact parenthesis, bracket, and causal arrow skeleton of Condition B, but randomly permutes Sacred symbol names across messages using a seeded derangement (`build_shuffle_control_map()`). If Condition B outperforms Condition D significantly, comprehension is proven to arise from the lexical semantics of the Sacred symbol names rather than superficial structural cues.
- **Condition E (Plaintext Upper Bound):** Replaces the wire with a concise one-sentence natural-language summary of the scenario, establishing the ceiling accuracy of the evaluated LLM on the 4-choice task.

Protocol Header Isolation (`--keep-ctx`), 4-Way Option Rotation (`--rotate-options`), and Paired McNemar Testing: To prevent context-header leakage, `eval_comprehension.py` defaults to **Skeletal Mode** (`@CTX` and `[k:v]` removed). When `--keep-ctx` is specified (**Protocol Structure Mode**, retaining `@CTX:DOMAIN` and masked payload keys `[pair:*, stop:*]`), outputs are written to separate `*_ctx.txt` / `*_ctx.json` files and scored independently. To eliminate LLM positional letter bias (e.g., preferring option B or C), `--rotate-options` replicates each item across all 4 cyclic permutations of options A, B, C, and D ($4N$ trials) and logs the chosen-letter histogram. Proportions are reported with **Wilson 95% confidence intervals**:

$$\mathrm{CI}_{\mathrm{Wilson}}(k, n, z = 1.96) = \frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}, \qquad \hat{p} = \frac{k}{n} \tag{2}$$

and paired condition comparisons (`A vs B`, `B vs C`, `B vs D`, `C vs E`) are evaluated via the **exact two-sided McNemar test** on discordant pairs (n_{01}, n_{10}): $p_{\mathrm{exact}} = \min\Big(1,\, 2\sum_{i=0}^{\min(n_{01},n_{10})} \binom{n_{01}+n_{10}}{i} 0.5^{n_{01}+n_{10}}\Big)$, alongside the marginal accuracy gain per +1,000 BPE tokens: $\Delta\mathrm{Acc}/(\Delta\mathrm{Tok}/1000)$.

6.2 Provisional Blind Comprehension Results ($n = 23$) and Stratified Domain Analysis

Table 7 presents the empirical results of the initial blind 4-choice comprehension evaluation ($n = 23$ questions, random guess baseline = 25.0%) conducted by Claude prior to corpus repair. In accordance with our scientific reporting standards, these figures are labeled as a **provisional baseline** (because the evaluator had partial prior exposure to the repository within the auditing session and sample size $n = 23$ yields wide confidence intervals):

- **Overall Accuracy Gain from Sacred Expansion:** On bare compact wire codes (Condition A), accuracy is $12/23 = \mathbf{52.2}\%$ (52%, Wilson 95% CI: [33.0%, 70.8%]). Expanding opcodes to Sacred identifiers (Condition B) raises accuracy to $16/23 = \mathbf{69.6}\%$ (70%, Wilson 95% CI: [49.1%, 84.4%]), an absolute gain of +17.4 percentage points at a marginal cost of only +26.5 BPE tokens/message (+6.57% accuracy gain per +100 tokens).
- **Stratified Routine Recovery vs. Non-Recovery Domains:** Stratifying the 23 items by domain reveals a sharp dichotomy. On **Routine Fault/Cluster Recovery** items ($n = 7$), both Condition A and Condition B achieve **7/7 = 100.0%** (Wilson 95% CI: [64.6%, 100.0%]) because fault-recovery chains exhibit distinctive structural signatures (`!->`, `?`) and unambiguous symbol names (`PROTO_GRACEFUL_DRAIN`, `SAFE_FALLBACK`). Conversely, on **Non-Recovery Specialized Domains** ($n = 16$: FX hedging, parser debugging, latent sync, hypothesis verification), Condition A drops to $5/16 = \mathbf{31.2}\%$ (31%, 95% CI: [14.2%, 55.6%], statistically indistinguishable from 25% chance), whereas Condition B rises to $9/16 = \mathbf{56.2}\%$ (56%, 95% CI: [33.2%, 76.9%]).
- **Root Cause of Non-Recovery Errors Fixed in v5.4.3:** Error analysis of the 7 missed items in Condition B revealed that 5 of the 7 failures were directly caused by the pre-fix corpus misalignments documented in Section 4.2 (e.g., Q3 and Q9 labeled as FX hedging while containing only recovery primitives, Q18 labeled as parser debugging while containing `ARBITRAGE` macro `§X64`, and Q11/Q16 containing the `GAiar` typo). With `corpus/dialogues.jsonl` now 100.0% aligned (1,606/1,606) and `eval_comprehension.py` supporting $N \geq 100$ holdout generation across Conditions A–E, multi-model holdout evaluation can be executed deterministically via a single CLI command (Section 8).

7. Threat Model and Unobservable EUF-CMA Proof of Origin Attestation

7.1 Unobservable Internal-Counter Attestation Construction (Zero Timestamp Jitter)

To prevent an active Dolev-Yao adversary $\mathcal{A}$ from injecting forged inter-agent commands, replaying stale packets, or observing rejection-sampling retries under an adaptive Chosen-Message Attack (EUF-CMA), TOAI-elegant-lang embeds a cryptographic origin tag directly into the 4-digit causal cost mantissa $c_n \in [0.0000, 0.9999]$. Let $S_0 \in \{0,1\}^{256}$ be the ephemeral

Table 7: Receiver Blind Comprehension Results (Claude Provisional Benchmark $n = 23$, 4-Choice Chance = 25.0%) and Harness Specifications

Evaluation Condition	Mean Prompt Tok	Overall Acc. ($n = 23$) [Wilson 95% CI]	Routine Recovery ($n = 7$) [95% CI]	Non-Recovery ($n = 16$) [95% CI]	Role in `eval_comprehension.py` (v5.4.3)
Cond A: Compact Wire Only (`§S01..§X64`)	118.4 tok	**52.2% (12/23)** [33.0%, 70.8%]	**100.0% (7/7)** [64.6%, 100.0%]	**31.2% (5/16)** [14.2%, 55.6%]	Baseline zero-shot test of opaque compact opcodes + operator skeleton (no `@CTX`, no `[k:v]`).
Cond B: Sacred Expanded (`§GAIA_WARMTH_*`)	144.9 tok	**69.6% (16/23)** [49.1%, 84.4%]	**100.0% (7/7)** [64.6%, 100.0%]	**56.2% (9/16)** [33.2%, 76.9%]	Tests zero-shot comprehension of deterministic Sacred symbol expansion (+17.4% vs. Cond A).
Cond C: Sacred + Rosetta Glossary Definitions	248.6 tok	Holdout Protocol ($N \geq 100$)	Holdout Protocol ($N \geq 100$)	Holdout Protocol ($N \geq 100$)	Injects `core_lexicon.json` & `macro_lexicon.json` definitions for every symbol in packet.
Cond D: Shuffle Control (Permuted Symbol Names)	144.9 tok	Holdout Protocol ($N \geq 100$)	Holdout Protocol ($N \geq 100$)	Holdout Protocol ($N \geq 100$)	Negative control: preserves syntax skeleton while permuting symbol names across items.
Cond E: Concise Plaintext Summary (Upper Bound)	136.2 tok	Holdout Protocol ($N \geq 100$)	Holdout Protocol ($N \geq 100$)	Holdout Protocol ($N \geq 100$)	Positive control: natural-language scenario summary establishing receiver model ceiling.

session key established via X25519 ECDH and HKDF-SHA256 (re-keyed every 2^{16} turns or upon `!-> §F04`, guaranteeing forward secrecy in volatile memory), $H_0 = \text{SHA256}(\texttt{"TOAI_TELEPATH_GENESIS_V541"} \,\|\, \text{SHA256}(\texttt{core_lexicon.json}) \,\|\, \text{SHA256}(\texttt{macro_lexicon.json}))$, and $H_{n-1} = \text{SHA256}(H_{n-2} \,\|\, \texttt{0x00} \,\|\, \text{wire}_{n-1})$. For any public turn timestamp t_n (subject to strict monotonicity $t_n > t_{n-1}$ enforced by the receiver), both sender and verifier deterministically compute the canonical 16-bit value $Y_n \in \{0, \ldots, 59{,}999\}$ for public confidence mantissa $P_n = \lfloor p_n \times 10^4 \rfloor \in \{0, \ldots, 9999\}$ via an **internal, non-transmitted rejection counter** $r_n \in \{0, 1, 2, \ldots\}$ (where $\text{dom}(t_n, r) = t_n$ for $r = 0$ and $t_n \,\|\, \text{uint8}(r)$ for $r \geq 1$):

$$r_n = \min\Big\{ r \in \mathbb{N}_0 : \big(P_n \oplus \text{trunc}_{16}(\text{HMAC-SHA256}(S_0,\, H_{n-1} \,\|\, \texttt{0x00} \,\|\, \text{dom}(t_n, r)))\big) < 60{,}000 \Big\} \tag{3}$$

and set $T_n = \text{trunc}_{16}(\text{HMAC-SHA256}(S_0,\, H_{n-1} \,\|\, \texttt{0x00} \,\|\, \text{dom}(t_n, r_n)))$. Because bitwise XOR $T \mapsto P_n \oplus T$ is a bijection on $\{0, \ldots, 65{,}535\}$, each candidate $Y_n^{(r)} = P_n \oplus T_n^{(r)}$ is uniform on $\{0, \ldots, 65{,}535\}$ with acceptance probability $60{,}000/65{,}536 = 91.55\%$ (expected internal loop count $\mathbb{E}[r_n + 1] = 1.092$ HMAC calls; for $r_n = 0$, including Table 8 where $Y_1 = 9841 \oplus 9247 = 622 < 60{,}000$, it takes 1 call). Given quantized 4-digit integers $P_n = \lfloor p_n \times 10^4 \rfloor \in \{0, \ldots, 9999\}$ and $C_n = \lfloor c_n \times 10^4 \rfloor \in \{0, \ldots, 9999\}$, verification is defined with explicit operator precedence (16-bit bitwise XOR precedes modulo 10,000 reduction):

$$\big((P_n \oplus T_n) \bmod 10{,}000\big) = C_n \tag{4}$$

7.2 Complete Case Analysis, Unrejected Bias Proof, and Timing Side-Channel Immunity

Theorem 1 (Exact EUF-CMA Forgery Bound Subsuming Direct Tag Guessing and Modular Aliases). *Under the $(\tau, Q + k, \epsilon_{PRF})$-PRF security of HMAC-SHA256 and ϵ_{coll}-collision resistance of SHA-256, for any adaptive EUF-CMA adversary $\mathcal{A}$ (even one that chooses or forces exact timestamps $t^*_{n+j} > t^*_{n+j-1}$), the probability of outputting $k \geq 1$ valid unqueried turns is bounded by:*

$$P^{\text{internal-rej}}_{\text{forge}}(k) \leq \left(\frac{6}{60{,}000}\right)^k + \epsilon_{\text{PRF}} + \epsilon_{\text{coll}} = 10^{-4k} + \epsilon_{\text{PRF}} + \epsilon_{\text{coll}} \quad \left(\text{and } P^{\text{unrejected}}_{\text{forge}}(k) \leq \left(\frac{7}{65{,}536}\right)^k + \epsilon_{\text{PRF}} + \epsilon_{\text{coll}}\right) \tag{5}$$

Proof. After eliminating SHA-256 chain collisions (Game_1, loss $\leq \epsilon_{\text{coll}}$) and replacing HMAC-SHA256$(S_0, \cdot)$ with a lazy uniform random function $\mathcal{R}$ (Game_2, loss $\leq \epsilon_{\text{PRF}}$), every tuple $(H^*_{n+j-1}, t^*_{n+j}, r)$ for an unqueried monotonic turn $t^*_{n+j} > t^*_{n+j-1}$ is strictly fresh, even when $\mathcal{A}$ chooses t^*_{n+j} and P^*_{n+j} adaptively. We analyze all candidate cost submissions C^*_{n+j} via two exhaustive cases:

- **Case 1 ($C^*_{n+j} \notin \{0, \ldots, 9999\}$ – Deterministic Syntactic Rejection):** By the LL(1) EBNF terminal production `cost_mantissa` (Section 3.2), any non-4-digit or out-of-range cost value is deterministically rejected by the lexer prior to MAC evaluation with probability $\Pr[\text{Accept} \mid C^*_{n+j} \notin \{0, \ldots, 9999\}] = 0$.
- **Case 2 ($C^*_{n+j} \in \{0, \ldots, 9999\}$ – Exact Congruence Cardinality $|\mathcal{W}(C^*_{n+j})| = 6$):** Because $T \mapsto P^*_{n+j} \oplus T$ is a bijection on $\{0, \ldots, 65{,}535\}$, the first internally accepted value $Y_{n+j} = P^*_{n+j} \oplus T_{n+j} \in \{0, \ldots, 59{,}999\}$ is strictly uniform over $\{0, \ldots, 59{,}999\}$. For any $C^*_{n+j} \in \{0, \ldots, 9999\}$, the winning congruence set $\mathcal{W}(C^*_{n+j}) = \{y \in \{0, \ldots, 59{,}999\} : y \equiv C^*_{n+j} \pmod{10{,}000}\} = \{C^*_{n+j} + 10{,}000m : m \in \{0, 1, 2, 3, 4, 5\}\}$ has cardinality $|\mathcal{W}(C^*_{n+j})| = 6$ (since $C^*_{n+j} + 50{,}000 \leq 59{,}999 < 60{,}000 \leq C^*_{n+j} + 60{,}000$), subsuming *Direct Tag Guessing* ($m = 0$, prob 1/60,000) and *Modular Aliases* ($m \in \{1, \ldots, 5\}$, prob 5/60,000), yielding $\Pr[\text{Accept}] = 6/60{,}000 = 10^{-4}$ per turn. Without rejection sampling ($Y_{n+j} \in \{0, \ldots, 65{,}535\}$), residues $C^*_{n+j} \in \{0, \ldots, 5{,}535\}$ admit a 7th alias ($m = 6$, since $C^*_{n+j} + 60{,}000 \leq 65{,}535$), bounding single-turn forgery by $\max_{C^*} |\mathcal{W}_{\text{unrej}}(C^*)|/65{,}536 = 7/65{,}536 \approx 1.068 \times 10^{-4}$.
- **Timing Unobservability:** Because neither r_n nor rejected candidates are transmitted, and one additional in-cache SHA-256 compression takes ≈ 150 ns on modern CPUs ($\mathbb{E}[r_n] \cdot 150$ ns $= 0.092 \times 150$ ns ≈ 0.014 μs), retry variance is > 6 orders of magnitude below network/LLM inference jitter (10^4–10^5 μs), leaking zero observable side-channel information. □

8. Reproducibility Artifact & Self-Contained Verification Commands

All source code, lexicons (`core_lexicon.json`, `macro_lexicon.json`, `rosetta_layer.json`), the 1,606-record aligned corpus (`corpus/dialogues.jsonl`), the 60-item stratified human audit sample (`corpus/human_audit_sample.csv`), and all benchmark suites are released under the MIT License at https://github.com/PhenoX-AI-Alliance/TOAI-elegant-lang and on PyPI (`pip install toai-elegant-lang==5.4.3`). Any reviewer can verify every claim, table, and cryptographic test vector in this paper on a standard CPU using Python 3.10+ standard library modules:

Table 8: Reproducible Cryptographic Test Vector for Equation (4) (Verifiable via Python `hashlib`/`hmac`, $r_1 = 0$)

Parameter / Verification Step	Canonical Hexadecimal / Decimal Value (Bitwise XOR Precedes mod 10,000)
Secret Seed S_0 (256-bit hex)	`7f83b1657ff1fc53b92dc18148a1d65dfc2d4b1fa3d677284addd200126d9069`
Session Anchor H_0 (SHA256)	`f3c515b25afe4f7ed4ca735b85fbfe50b083c7af4b8ec8c107c0e99438f02998` (`SHA256(b"TOAI_TELEPATH_GENESIS_V541")`)
Turn-1 Timestamp t_1 (UTF-8, $r_1 = 0$)	`2026-11-06T12:00:00Z` (HMAC-SHA256($S_0, H_0 \,||\, $ `b"\x002026-11-06T12:00:00Z"`) $\rightarrow T_1 =$ `0x241f` $= \mathbf{9247}$)
Step 1: 16-Bit Bitwise XOR	Given $p_1 = 0.9841 \implies P_1 = 9841$: $Y_1 = (P_1 \oplus T_1) = (9841 \oplus 9247) = \mathbf{622} < 60{,}000$ ($r_1 = 0$ accepted)
Step 2: Modulo 10,000 Reduction	$C_1 = Y_1 \bmod 10{,}000 = 622 \bmod 10{,}000 = \mathbf{622} \implies c_1 = C_1/10^4 = \mathbf{0.0622}$

```
# 1. Install canonical release v5.4.3 from PyPI (or clone repository) and run all 20 unit tests:
pip install toai-elegant-lang==5.4.3
python3 -m unittest discover -s tests -v && python3 -m unittest src/test_telepath_integrity.py -v

# 2. Reproduce Table 5 & Table 6 (Fair 5-Format BPE Benchmark & 1,000-Token Header Break-Even Analysis):
python3 benchmarks/fair_bench.py

# 3. Reproduce Table 4 & Section 4.2 (Independent Non-Circular Corpus Alignment & Human Audit Cohen's kappa = 0.8611):
python3 src/audit_corpus_alignment.py
python3 src/audit_corpus_alignment.py --score-sample corpus/human_audit_sample.csv

# 4. Build & score 5-Condition Receiver Comprehension Prompts (Conditions A-E, Wilson 95% CIs, McNemar Test):
python3 benchmarks/eval_comprehension.py build --cond ALL -n 100 --seed 42
python3 benchmarks/eval_comprehension.py build --cond ALL -n 24 --keep-ctx --rotate-options

# 5. Self-Contained One-Line Verification of Codec Round-Trip, Quote-Aware Payload, OOV Policy, and Table 8 HMAC Vector:
python3 -c '
import hashlib, hmac, sys; sys.path.insert(0, "src"); from codec import ElegantCodec, UnknownSymbolError
c = ElegantCodec()
stmt = "@CTX:FX_HEDGE :: §X64 [pair:\"USD/JPY\", code:\"007\", note:\"a, b\", expr:\"x]y\", stop:150.5] -[p:0.99, c:0.01]-> §PROTO_HEALTH_ATTEST => §
      GENTLE_RESOURCE_SHARING"
wire = str(c.encode(stmt, on_oov="error")); p = c.extract_payloads(wire)[0]
assert c.decode(wire, expand_macros=False) == stmt and p["code"] == "007" and p["note"] == "a, b" and p["expr"] == "x]y"
S0 = bytes.fromhex("7f83b1657ff1fc53b92dc18148a1d65dfc2d4b1fa3d677284addd200126d9069")
H0 = hashlib.sha256(b"TOAI_TELEPATH_GENESIS_V541").digest()
T1 = int.from_bytes(hmac.new(S0, H0 + b"\x002026-11-06T12:00:00Z", hashlib.sha256).digest()[:2], "big")
assert T1 == 9247 and ((9841 ^ T1) % 10000) == 622
print("ALL VERIFIED: Lossless AST Round-Trip, Quote-Aware Payload, and EUF-CMA Vector (T1=9247, C1=622)")
'
```

References and Verifiable Artifacts

[1] Anthropic. (2024). *Model Context Protocol (MCP): Open Specification for JSON-RPC 2.0 Host-Client-Server Context Exchange.* https://modelcontextprotocol.io/

[2] Google Cloud & Agent Ecosystem Contributors. (2025). *Agent2Agent (A2A) Protocol Specification: Open Interoperability Standard for Multi-Agent Systems.* https://github.com/google/A2A

[3] Foundation for Intelligent Physical Agents (FIPA). (2002). *FIPA ACL Message Structure Specification (SC00061G).* IEEE Computer Society Standards Committee.

[4] Jiang, H., Wu, Q., Lin, C.-Y., Yang, Y., & Qiu, L. (2023). LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 13358–13376. https://aclanthology.org/2023.emnlp-main.825/

[5] Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awedat, A. H., White, R. W., Burger, D., & Wang, C. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155.*

[6] Foerster, J., Assael, I. A., de Freitas, N., & Whiteson, S. (2016). Learning to Communicate with Deep Multi-Agent Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 29, pp. 2137–2145.

[7] Lazaridou, A., Peysakhovich, A., & Baroni, M. (2017). Multi-Agent Cooperation and the Emergence of (Natural) Language. In *International Conference on Learning Representations (ICLR).*

[8] Google. (2008). *Protocol Buffers: Language-Neutral, Platform-Neutral Extensible Mechanism for Serializing Structured Data.* https://protobuf.dev/

[9] Wilson, E. B. (1927). Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 22(158), 209–212.

[10] McNemar, Q. (1947). Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika*, 12(2), 153–157.

[11] Cohen, J. (1960). A Coefficient of Agreement for Nominal Scales. *Educational and Psychological Measurement*, 20(1), 37–46.

[12] Toriyama, K. (PhenoX). (2026). *toai-elegant-lang v5.4.3: Zero-Dependency Tri-Script Orthogonal RISC Wire Protocol, Non-Circular Corpus Auditor, and Empirical Comprehension Harness.* GitHub & Python Package Index (PyPI). https://github.com/PhenoX-AI-Alliance/TOAI-elegant-lang | https://pypi.org/project/toai-elegant-lang/5.4.3/